SAC

Soft Actor Critic (SAC) Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor.

SAC is the successor of Soft Q-Learning SQL and incorporates the double Q-learning trick from TD3. A key feature of SAC, and a major difference with common RL algorithms, is that it is trained to maximize a trade-off between expected return and entropy, a measure of randomness in the policy.

Available Policies

`MlpPolicy`	
`CnnPolicy`	Policy class (with both actor and critic) for SAC.
`MultiInputPolicy`	Policy class (with both actor and critic) for SAC.

Notes

- Original paper: https://arxiv.org/abs/1801.01290
- OpenAI Spinning Guide for SAC: https://spinningup.openai.com/en/latest/algorithms/sac.html
- Original Implementation: https://github.com/haarnoja/sac
- Blog post on using SAC with real robots: https://bair.berkeley.edu/blog/2018/12/14/sac/

Note

In our implementation, we use an entropy coefficient (as in OpenAI Spinning or Facebook Horizon), which is the equivalent to the inverse of reward scale in the original SAC paper. The main reason is that it avoids having too high errors when updating the Q functions.

master

AI-powered ad network for devs. Get your message in front of the right developers with EthicalAds.

Ads by EthicalAds

Close Ad

When automatically adjusting the temperature (alpha/entropy coefficient), we optimize the logarithm of the entropy coefficient instead of the entropy coefficient itself. This is consistent with the original implementation and has proven to be more stable (see issues GH#36, #55 and others).

Note

The default policies for SAC differ a bit from others MlpPolicy: it uses ReLU instead of tanh activation, to match the original paper

Can I use?

- Recurrent policies: ❌
- Multi processing: ✔
- Gym spaces:

Space	Action	Observation
Discrete	❌	✔
Box	✔	✔
MultiDiscrete	❌	✔
MultiBinary	❌	✔
Dict	❌	✔

Example

This example is only to demonstrate the use of the library and its functions, and the trained agents may not solve the environments. Optimized hyperparameters can be found in RL Zoo repository.

```python
import gymnasium as gym

from stable_baselines3 import SAC

env = gym.make("Pendulum-v1", render_mode="human")

model = SAC("MlpPolicy", env, verbose=1)
model.learn(total_timesteps=10000, log_interval=4)
model.save("sac_pendulum")

del model # remove to demonstrate saving and loading

model = SAC.load("sac_pendulum")

obs, info = env.reset()
while True:
    action, _states = model.predict(obs, deterministic=True)
    obs, reward, terminated, truncated, info = env.step(action)
    if terminated or truncated:
        obs, info = env.reset()
```

Note

Using gSDE (Generalized State-Dependent Exploration) during inference (see PR #1767):

When using SAC models trained with `use_sde=True`, the automatic noise resetting that occurs during training (controlled by `sde_sample_freq`) does not happen when using `model.predict()` for inference. This results in deterministic behavior even when `deterministic=False`.

For continuous control tasks, it is recommended to use deterministic behavior during inference (`deterministic=True`). If you need stochastic behavior during inference, you must manually reset the noise by calling `model.policy.reset_noise(env.num_envs)` at appropriate intervals based on your desired `sde_sample_freq`.

Results

PyBullet Environments

Results on the PyBullet benchmark (1M steps) using 3 seeds. The complete learning curves are available in the associated issue #48.

Note

Hyperparameters from the gSDE paper were used (as they are tuned for PyBullet envs).

Gaussian means that the unstructured Gaussian noise is used for exploration, *gSDE* (generalized State-Dependent Exploration) is used otherwise.

Environments	SAC	SAC	TD3
	Gaussian	gSDE	Gaussian
HalfCheetah	2757 +/- 53	2984 +/- 202	2774 +/- 35
Ant	3146 +/- 35	3102 +/- 37	3305 +/- 43
Hopper	2422 +/- 168	2262 +/- 1	2429 +/- 126
Walker2D	2184 +/- 54	2136 +/- 67	2063 +/- 185

How to replicate the results?

Clone the rl-zoo repo:

```
git clone https://github.com/DLR-RM/rl-baselines3-zoo
cd rl-baselines3-zoo/
```

Run the benchmark (replace `$ENV_ID` by the envs mentioned above):

```
python train.py --algo sac --env $ENV_ID --eval-episodes 10 --eval-freq 10000
```

Plot the results:

```
python scripts/all_plots.py -a sac -e HalfCheetah Ant Hopper Walker2D -f logs/ -o
logs/sac_results
python scripts/plot_from_file.py -i logs/sac_results.pkl -latex -l SAC
```

Parameters

class **stable_baselines3.sac.SAC**(*policy*, *env*, *learning_rate=0.0003*, *buffer_size=1000000*, *learning_starts=100*, *batch_size=256*, *tau=0.005*, *gamma=0.99*, *train_freq=1*, *gradient_steps=1*, *action_noise=None*, *replay_buffer_class=None*, *replay_buffer_kwargs=None*, *optimize_memory_usage=False*, *n_steps=1*, *ent_coef='auto'*, *target_update_interval=1*, *target_entropy='auto'*, *use_sde=False*, *sde_sample_freq=-1*, *use_sde_at_warmup=False*, *stats_window_size=100*, *tensorboard_log=None*, *policy_kwargs=None*, *verbose=0*, *seed=None*, *device='auto'*, *_init_setup_model=True*) [source]

Soft Actor-Critic (SAC) Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor, This implementation borrows code from original implementation (https://github.com/haarnoja/sac) from OpenAI Spinning Up (https://github.com/openai/spinningup), from the softlearning repo (https://github.com/rail-berkeley/softlearning/) and from Stable Baselines (https://github.com/hill-a/stable-baselines) Paper: https://arxiv.org/abs/1801.01290 Introduction to SAC: https://spinningup.openai.com/en/latest/algorithms/sac.html

Note: we use double q target and not value target as discussed in https://github.com/hill-a/stable-baselines/issues/270

Parameters:

- **policy** (*SACPolicy*) – The policy model to use (MlpPolicy, CnnPolicy, ...)
- **env** (*Env* | *VecEnv* | *str*) – The environment to learn from (if registered in Gym, can be str)
- **learning_rate** (*float* | *Callable[[float], float]*) – learning rate for adam optimizer, the same learning rate will be used for all networks (Q-Values, Actor and Value function) it can be a function of the current progress remaining (from 1 to 0)
- **buffer_size** (*int*) – size of the replay buffer
- **learning_starts** (*int*) – how many steps of the model to collect transitions for before learning starts
- **batch_size** (*int*) – Minibatch size for each gradient update
- **tau** (*float*) – the soft update coefficient ("Polyak update", between 0 and 1)
- **gamma** (*float*) – the discount factor
- **train_freq** (*int* | *tuple[int, str]*) – Update the model every `train_freq` steps. Alternatively pass a tuple of frequency and unit like `(5, "step")` or `(2, "episode")`.
- **gradient_steps** (*int*) – How many gradient steps to do after each rollout (see `train_freq`) Set to `-1` means to do as many gradient steps as steps done in the environment during the rollout.

- **action_noise** (*ActionNoise* | *None*) – the action noise type (None by default), this can help for hard exploration problem. Cf common.noise for the different action noise type.
- **replay_buffer_class** (*type[ReplayBuffer]* | *None*) – Replay buffer class to use (for instance `HerReplayBuffer`). If `None`, it will be automatically selected.
- **replay_buffer_kwargs** (*dict[str, Any]* | *None*) – Keyword arguments to pass to the replay buffer on creation.
- **optimize_memory_usage** (*bool*) – Enable a memory efficient variant of the replay buffer at a cost of more complexity. See https://github.com/DLR-RM/stable-baselines3/issues/37#issuecomment-637501195
- **n_steps** (*int*) – When n_step > 1, uses n-step return (with the NStepReplayBuffer) when updating the Q-value network.
- **ent_coef** (*str* | *float*) – Entropy regularization coefficient. (Equivalent to inverse of reward scale in the original SAC paper.) Controlling exploration/exploitation trade-off. Set it to 'auto' to learn it automatically (and 'auto_0.1' for using 0.1 as initial value)
- **target_update_interval** (*int*) – update the target network every `target_network_update_freq` gradient steps.
- **target_entropy** (*str* | *float*) – target entropy when learning `ent_coef` (`ent_coef = 'auto'`)
- **use_sde** (*bool*) – Whether to use generalized State Dependent Exploration (gSDE) instead of action noise exploration (default: False)
- **sde_sample_freq** (*int*) – Sample a new noise matrix every n steps when using gSDE Default: -1 (only sample at the beginning of the rollout)
- **use_sde_at_warmup** (*bool*) – Whether to use gSDE instead of uniform sampling during the warm up phase (before learning starts)
- **stats_window_size** (*int*) – Window size for the rollout logging, specifying the number of episodes to average the reported success rate, mean episode length, and mean reward over
- **tensorboard_log** (*str* | *None*) – the log location for tensorboard (if None, no logging)
- **policy_kwargs** (*dict[str, Any]* | *None*) – additional arguments to be passed to the policy on creation. See SAC Policies

- **verbose** (*int*) – Verbosity level: 0 for no output, 1 for info messages (such as device or wrappers used), 2 for debug messages
- **seed** (*int* | *None*) – Seed for the pseudo random generators
- **device** (*device* | *str*) – Device (cpu, cuda, ...) on which the code should be run. Setting it to auto, the code will be run on the GPU if possible.
- **_init_setup_model** (*bool*) – Whether or not to build the network at the creation of the instance

`collect_rollouts`(*env*, *callback*, *train_freq*, *replay_buffer*, *action_noise=None*, *learning_starts=0*, *log_interval=None*)

Collect experiences and store them into a `ReplayBuffer`.

Parameters:
- **env** (*VecEnv*) – The training environment
- **callback** (*BaseCallback*) – Callback that will be called at each step (and at the beginning and end of the rollout)
- **train_freq** (*TrainFreq*) – How much experience to collect by doing rollouts of current policy. Either `TrainFreq(<n>, TrainFrequencyUnit.STEP)` or `TrainFreq(<n>, TrainFrequencyUnit.EPISODE)` with `<n>` being an integer greater than 0.
- **action_noise** (*ActionNoise* | *None*) – Action noise that will be used for exploration Required for deterministic policy (e.g. TD3). This can also be used in addition to the stochastic policy for SAC.
- **learning_starts** (*int*) – Number of steps before learning for the warm-up phase.
- **replay_buffer** (*ReplayBuffer*)
- **log_interval** (*int* | *None*) – Log data every `log_interval` episodes

Returns:

Return type: *RolloutReturn*

`dump_logs`()

Write log data.

Return type: None

`get_env`()

Returns the current environment (can be None if not defined).

Returns: The current environment
Return type: *VecEnv* | None

`get_parameters`()

Return the parameters of the agent. This includes parameters from different networks, e.g. critics (value functions) and policies (pi functions).

Returns: Mapping of from names of the objects to PyTorch state-dicts.
Return type: dict[str, dict]

`get_vec_normalize_env`()

Return the `VecNormalize` wrapper of the training env if it exists.

Returns: The `VecNormalize` env.
Return type: *VecNormalize* | None

`learn`(*total_timesteps*, *callback=None*, *log_interval=4*, *tb_log_name='SAC'*, *reset_num_timesteps=True*, *progress_bar=False*) [source]

Return a trained model.

Parameters:

- **total_timesteps** (*int*) – The total number of samples (env steps) to train on Note: it is a lower bound, see issue #1150
- **callback** (*None* | *Callable* | *list[BaseCallback]* | *BaseCallback*) – callback(s) called at every step with state of the algorithm.
- **log_interval** (*int*) – for on-policy algos (e.g., PPO, A2C, ...) this is the number of training iterations (i.e., log_interval * n_steps * n_envs timesteps) before logging; for off-policy algos (e.g., TD3, SAC, ...) this is the number of episodes before logging.
- **tb_log_name** (*str*) – the name of the run for TensorBoard logging
- **reset_num_timesteps** (*bool*) – whether or not to reset the current timestep number (used in logging)
- **progress_bar** (*bool*) – Display a progress bar using tqdm and rich.
- **self** (*SelfSAC*)

Returns: the trained model

Return type: *SelfSAC*

classmethod **load**(*path*, *env=None*, *device='auto'*, *custom_objects=None*, *print_system_info=False*, *force_reset=True*, ***kwargs*)

Load the model from a zip-file. Warning: `load` re-creates the model from scratch, it does not update it in-place! For an in-place load use `set_parameters` instead.

Parameters:
- **path** (*str* | *Path* | *BufferedIOBase*) – path to the file (or a file-like) where to load the agent from
- **env** (*Env* | *VecEnv* | *None*) – the new environment to run the loaded model on (can be None if you only need prediction from a trained model) has priority over any saved environment
- **device** (*device* | *str*) – Device on which the code should run.
- **custom_objects** (*dict[str, Any]* | *None*) – Dictionary of objects to replace upon loading. If a variable is present in this dictionary as a key, it will not be deserialized and the corresponding item will be used instead. Similar to custom_objects in `keras.models.load_model`. Useful when you have an object in file that can not be deserialized.
- **print_system_info** (*bool*) – Whether to print system info from the saved model and the current system info (useful to debug loading issues)
- **force_reset** (*bool*) – Force call to `reset()` before training to avoid unexpected behavior. See https://github.com/DLR-RM/stable-baselines3/issues/597
- **kwargs** – extra arguments to change the model when loading

Returns: new model instance with loaded parameters

Return type: *SelfBaseAlgorithm*

load_replay_buffer(*path*, *truncate_last_traj=True*)

Load a replay buffer from a pickle file.

Parameters:
- **path** (*str* | *Path* | *BufferedIOBase*) – Path to the pickled replay buffer.
- **truncate_last_traj** (*bool*) – When using `HerReplayBuffer` with online sampling: If set to `True`, we assume that the last trajectory in the replay buffer was finished (and truncate it).

If set to `False`, we assume that we continue the same trajectory (same episode).

Return type: None

property **logger**: Logger

Getter for the logger object.

predict(*observation*, *state=None*, *episode_start=None*, *deterministic=False*)

Get the policy action from an observation (and optional hidden state). Includes sugar-coating to handle different observations (e.g. normalizing images).

Parameters:
- **observation** (*ndarray* | *dict[str, ndarray]*) – the input observation
- **state** (*tuple[ndarray, ...]* | *None*) – The last hidden states (can be None, used in recurrent policies)
- **episode_start** (*ndarray* | *None*) – The last masks (can be None, used in recurrent policies) this correspond to beginning of episodes, where the hidden states of the RNN must be reset.
- **deterministic** (*bool*) – Whether or not to return deterministic actions.

Returns: the model's action and the next hidden state (used in recurrent policies)

Return type: tuple[*ndarray*, tuple[*ndarray*, ...] | None]

save(*path*, *exclude=None*, *include=None*)

Save all the attributes of the object and the model parameters in a zip-file.

Parameters:
- **path** (*str* | *Path* | *BufferedIOBase*) – path to the file where the rl agent should be saved
- **exclude** (*Iterable[str]* | *None*) – name of parameters that should be excluded in addition to the default ones
- **include** (*Iterable[str]* | *None*) – name of parameters that might be excluded but should be included anyway

Return type: None

save_replay_buffer(*path*)

Save the replay buffer as a pickle file.

Parameters: **path** (*str | Path | BufferedIOBase*) – Path to the file where the replay buffer should be saved. if path is a str or pathlib.Path, the path is automatically created if necessary.

Return type: None

`set_env`(*env*, *force_reset=True*)

Checks the validity of the environment, and if it is coherent, set it as the current environment. Furthermore wrap any non vectorized env into a vectorized checked parameters: - observation_space - action_space

Parameters:

- **env** (*Env | VecEnv*) – The environment for learning a policy
- **force_reset** (*bool*) – Force call to `reset()` before training to avoid unexpected behavior. See issue https://github.com/DLR-RM/stable-baselines3/issues/597

Return type: None

`set_logger`(*logger*)

Setter for for logger object.

Warning

When passing a custom logger object, this will overwrite `tensorboard_log` and `verbose` settings passed to the constructor.

Parameters: **logger** (*Logger*)

Return type: None

`set_parameters`(*load_path_or_dict*, *exact_match=True*, *device='auto'*)

Load parameters from a given zip-file or a nested dictionary containing parameters for different modules (see `get_parameters`).

Parameters:

- **load_path_or_iter** – Location of the saved data (path or file-like, see `save`), or a nested dictionary containing nn.Module parameters used by the policy. The dictionary maps object names to a state-dictionary returned by `torch.nn.Module.state_dict()`.
- **exact_match** (*bool*) – If True, the given parameters should include parameters for each module and each of their parameters, otherwise raises an Exception. If set to False,

this can be used to update only specific parameters.
- **device** (*device | str*) – Device on which the code should run.
- **load_path_or_dict** (*str | dict[str, Tensor]*)

Return type: None

set_random_seed(*seed=None*)

Set the seed of the pseudo-random generators (python, numpy, pytorch, gym, action_space)

Parameters: **seed** (*int | None*)
Return type: None

train(*gradient_steps*, *batch_size=64*) [source]

Sample the replay buffer and do the updates (gradient descent and update target networks)

Parameters:
- **gradient_steps** (*int*)
- **batch_size** (*int*)

Return type: None

SAC Policies

stable_baselines3.sac.MlpPolicy

alias of `SACPolicy`

class **stable_baselines3.sac.policies.SACPolicy**(*observation_space*, *action_space*, *lr_schedule*, *net_arch=None*, *activation_fn=<class 'torch.nn.modules.activation.ReLU'>*, *use_sde=False*, *log_std_init=-3*, *use_expln=False*, *clip_mean=2.0*, *features_extractor_class=<class 'stable_baselines3.common.torch_layers.FlattenExtractor'>*, *features_extractor_kwargs=None*, *normalize_images=True*, *optimizer_class=<class 'torch.optim.adam.Adam'>*, *optimizer_kwargs=None*, *n_critics=2*, *share_features_extractor=False*) [source]

Policy class (with both actor and critic) for SAC.

Parameters:
- **observation_space** (*Space*) – Observation space
- **action_space** (*Box*) – Action space
- **lr_schedule** (*Callable[[float], float]*) – Learning rate schedule (could be constant)
- **net_arch** (*list[int] | dict[str, list[int]] | None*) – The specification of the policy and value networks.
- **activation_fn** (*type[Module]*) – Activation function

- **use_sde** (*bool*) – Whether to use State Dependent Exploration or not
- **log_std_init** (*float*) – Initial value for the log standard deviation
- **use_expln** (*bool*) – Use `expln()` function instead of `exp()` when using gSDE to ensure a positive standard deviation (cf paper). It allows to keep variance above zero and prevent it from growing too fast. In practice, `exp()` is usually enough.
- **clip_mean** (*float*) – Clip the mean output when using gSDE to avoid numerical instability.
- **features_extractor_class** (*type[BaseFeaturesExtractor]*) – Features extractor to use.
- **features_extractor_kwargs** (*dict[str, Any] | None*) – Keyword arguments to pass to the features extractor.
- **normalize_images** (*bool*) – Whether to normalize images or not, dividing by 255.0 (True by default)
- **optimizer_class** (*type[Optimizer]*) – The optimizer to use, `th.optim.Adam` by default
- **optimizer_kwargs** (*dict[str, Any] | None*) – Additional keyword arguments, excluding the learning rate, to pass to the optimizer
- **n_critics** (*int*) – Number of critic networks to create.
- **share_features_extractor** (*bool*) – Whether to share or not the features extractor between the actor and the critic (this saves computation time)

forward(*obs*, *deterministic=False*) [source]

Define the computation performed at every call.

Should be overridden by all subclasses.

Note

Although the recipe for forward pass needs to be defined within this function, one should call the `Module` instance afterwards instead of this since the former takes care of running the registered hooks while the latter silently ignores them.

Parameters:
- **obs** (*Tensor | dict[str, Tensor]*)
- **deterministic** (*bool*)

Return type: *Tensor*

reset_noise(*batch_size=1*) [source]

Sample new weights for the exploration matrix, when using gSDE.

Parameters: **batch_size** (*int*)
Return type: None

set_training_mode(*mode*) [source]

Put the policy in either training or evaluation mode.

This affects certain modules, such as batch normalisation and dropout.

Parameters: **mode** (*bool*) – if true, set to training mode, else set to evaluation mode
Return type: None

---

class **stable_baselines3.sac.CnnPolicy**(*observation_space, action_space, lr_schedule, net_arch=None, activation_fn=<class 'torch.nn.modules.activation.ReLU'>, use_sde=False, log_std_init=-3, use_expln=False, clip_mean=2.0, features_extractor_class=<class 'stable_baselines3.common.torch_layers.NatureCNN'>, features_extractor_kwargs=None, normalize_images=True, optimizer_class=<class 'torch.optim.adam.Adam'>, optimizer_kwargs=None, n_critics=2, share_features_extractor=False*) [source]

Policy class (with both actor and critic) for SAC.

Parameters:
- **observation_space** (*Space*) – Observation space
- **action_space** (*Box*) – Action space
- **lr_schedule** (*Callable[[float], float]*) – Learning rate schedule (could be constant)
- **net_arch** (*list[int] | dict[str, list[int]] | None*) – The specification of the policy and value networks.
- **activation_fn** (*type[Module]*) – Activation function
- **use_sde** (*bool*) – Whether to use State Dependent Exploration or not
- **log_std_init** (*float*) – Initial value for the log standard deviation
- **use_expln** (*bool*) – Use `expln()` function instead of `exp()` when using gSDE to ensure a positive standard deviation (cf paper). It allows to keep variance above zero and prevent it from growing too fast. In practice, `exp()` is usually enough.
- **clip_mean** (*float*) – Clip the mean output when using gSDE to avoid numerical instability.
- **features_extractor_class** (*type[BaseFeaturesExtractor]*) – Features extractor to use.

- **normalize_images** (*bool*) – Whether to normalize images or not, dividing by 255.0 (True by default)
- **optimizer_class** (*type[Optimizer]*) – The optimizer to use, `th.optim.Adam` by default
- **optimizer_kwargs** (*dict[str, Any] | None*) – Additional keyword arguments, excluding the learning rate, to pass to the optimizer
- **n_critics** (*int*) – Number of critic networks to create.
- **share_features_extractor** (*bool*) – Whether to share or not the features extractor between the actor and the critic (this saves computation time)
- **features_extractor_kwargs** (*dict[str, Any] | None*)

---

class **stable_baselines3.sac.MultiInputPolicy**(*observation_space, action_space, lr_schedule, net_arch=None, activation_fn=<class 'torch.nn.modules.activation.ReLU'>, use_sde=False, log_std_init=-3, use_expln=False, clip_mean=2.0, features_extractor_class=<class 'stable_baselines3.common.torch_layers.CombinedExtractor'>, features_extractor_kwargs=None, normalize_images=True, optimizer_class=<class 'torch.optim.adam.Adam'>, optimizer_kwargs=None, n_critics=2, share_features_extractor=False*) [source]

Policy class (with both actor and critic) for SAC.

Parameters:

- **observation_space** (*Space*) – Observation space
- **action_space** (*Box*) – Action space
- **lr_schedule** (*Callable[[float], float]*) – Learning rate schedule (could be constant)
- **net_arch** (*list[int] | dict[str, list[int]] | None*) – The specification of the policy and value networks.
- **activation_fn** (*type[Module]*) – Activation function
- **use_sde** (*bool*) – Whether to use State Dependent Exploration or not
- **log_std_init** (*float*) – Initial value for the log standard deviation
- **use_expln** (*bool*) – Use `expln()` function instead of `exp()` when using gSDE to ensure a positive standard deviation (cf paper). It allows to keep variance above zero and prevent it from growing too fast. In practice, `exp()` is usually enough.
- **clip_mean** (*float*) – Clip the mean output when using gSDE to avoid numerical instability.
- **features_extractor_class** (*type[BaseFeaturesExtractor]*) – Features extractor to use.
- **normalize_images** (*bool*) – Whether to normalize images or not, dividing by 255.0 (True by default)

- **optimizer_class** (*type[Optimizer]*) – The optimizer to use, `th.optim.Adam` by default
- **optimizer_kwargs** (*dict[str, Any] | None*) – Additional keyword arguments, excluding the learning rate, to pass to the optimizer
- **n_critics** (*int*) – Number of critic networks to create.
- **share_features_extractor** (*bool*) – Whether to share or not the features extractor between the actor and the critic (this saves computation time)
- **features_extractor_kwargs** (*dict[str, Any] | None*)